Assignment 3 – Problem Based Learning

Name: Ahmad Sabhi.
Roll NO: Fa-24/BSCS/399.

Section: F

Problem 1: Translating C Code to Assembly

Objective

To understand how high-level C code is translated into low-level assembly instructions and how optimizations improve performance.

Assumption:
We use **generic x86-like assembly syntax** (simple, educational level).
Registers used: `AX, BX, CX, DX, SI`

Example 1: Basic Arithmetic Operations

C Code

```
int a = 5, b = 10, c;
c = a + b;
```

Assembly Translation

```
MOV AX, 5      ; Load value 5 into register AX (a)
MOV BX, 10     ; Load value 10 into register BX (b)
ADD AX, BX     ; AX = AX + BX
```

```
MOV CX, AX      ; Store result in c (CX)
```

Explanation

- Values are loaded directly into registers
- Arithmetic is done using registers (faster than memory)
- Result is stored in another register

Optimization

- No memory access after initialization
- Minimal instructions
- Highly efficient for simple arithmetic

Example 2: Loop Structure

C Code

```
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += i;
}
```

Assembly Translation

```
MOV CX, 0       ; sum = 0
MOV AX, 0       ; i = 0

LOOP_START:
CMP AX, 5       ; compare i with 5
JGE LOOP_END  ; if i >= 5, exit loop
ADD CX, AX      ; sum += i
INC AX          ; i++
JMP LOOP_START
```

```
LOOP_END:
```

Explanation

- AX holds loop variable `i`
- CX holds `sum`
- `CMP` and `JGE` control the loop
- `INC` is used instead of ADD for optimization

Optimization

- Registers only (no memory access)
- Simple branching
- Efficient loop control

Example 3: Array Manipulation

C Code

```
int arr[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++) {
    arr[i] = arr[i] * 2;
}
```

Assembly Translation

```
MOV SI, 0          ; i = 0

ARRAY_LOOP:
CMP SI, 5
JGE END_ARRAY

MOV AX, arr[SI]   ; load arr[i]
SHL AX, 1          ; multiply by 2 (faster than MUL)
MOV arr[SI], AX   ; store back
```

```
INC SI
JMP ARRAY_LOOP

END_ARRAY:
```

Explanation

- SI is used as array index
- SHL (shift left) multiplies by 2 efficiently
- Array accessed using indexed addressing

Optimization

- Shift instead of multiplication
- Minimal instructions
- Efficient memory access

Performance Comparison: C vs Assembly

Aspect	C Code	Assembly Code
Instruction control	Compiler dependent	Full control
Execution speed	Moderate	Faster
Memory access	More	Optimized
Register usage	Automatic	Explicit
Optimization level	Limited	High

Optimized assembly executes faster due to:

- Reduced instruction count
- Direct register usage
- Efficient branching

Problem 2: Cache Replacement Techniques & Pipelining

1. Cache Replacement Techniques (5)

1. Least Recently Used (LRU)

- Replaces the block not used for the longest time
- High hit rate
- Complex to implement

2. First In First Out (FIFO)

- Replaces oldest cache block
- Simple
- May remove frequently used data

3. Least Frequently Used (LFU)

- Removes block with lowest access count
- Good for stable workloads
- Poor for changing patterns

4. Random Replacement

- Randomly replaces a block
- Simple hardware
- Unpredictable performance

5. Most Recently Used (MRU)

- Replaces most recently accessed block
- Useful in looping workloads
- Less common

2. Impact on Pipelined Processors

Policy	Hit Rate	Pipeline Impact
LRU	Very High	Fewer stalls
FIFO	Medium	Moderate stalls
LFU	High	Stable pipeline
Random	Low	Frequent stalls
MRU	Medium	Workload dependent

3. Pipeline Analysis (5-Stage Pipeline)

Pipeline Stages

1. IF – Instruction Fetch
2. ID – Instruction Decode
3. EX – Execute
4. MEM – Memory Access
5. WB – Write Back

Sample Workload

- 20 instructions
- Cache size: 4 blocks
- Memory accesses: 10

LRU Example

- Cache hits: 8
- Cache misses: 2
- CPI ≈ **1.2**
- High throughput

FIFO Example

- Cache hits: 6
- Cache misses: 4
- CPI ≈ **1.5**

Random Example

- Cache hits: 5
- Cache misses: 5
- CPI ≈ **1.7**

Pipeline Hazards

1. Structural Hazards

- Same hardware resource needed
- **Mitigation:** Resource duplication

2. Data Hazards

- Instruction depends on previous result
- **Mitigation:** Forwarding, stalling

3. Control Hazards

- Branch instructions
- **Mitigation:** Branch prediction, delayed branching

Optimization Recommendations

- Use **LRU cache replacement** for high hit rate
- Apply **branch prediction** to reduce control hazards
- Use **forwarding paths** to reduce data hazards

- Increase cache size to reduce memory stalls

Conclusion

- Assembly translation gives deep understanding of processor behavior
- Optimized assembly improves execution speed
- Cache replacement policies significantly affect pipeline performance
- LRU provides best balance of performance and efficiency